Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт

Про виконання лабораторної роботи №4

“ОСНОВНІ ТИПИ ЗМІННИХ. ОПЕРАЦІЇ ТА ЇХ ПРІОРИТЕТ”

Виконав

Студент ФЕП-13

Троць А.Т

Перевірив

Ас. Васюта В.М

Львів 2026

Мета роботи: Вивчити поняття: основні типи змінних, вказівники, операції та їх пріоритет.

Обладнання та програмне забезпечення: – IBM сумісна персональна обчислювальна машина; – онлайн компілятор мови програмування Сі, доступний за посиланням https://repl.it/languages/c

Завдання до роботи: Написати програму для з використанням мови програмування Сі, котра реалізує наступні функції: – використання основних типів змінних; – реалізацію всіх операторів; – обчислення складних математичних виразів.

Хід роботи

Завдання 2:

Частина коду програми:

```c
#include <stdio.h>

void print_bin(unsigned char n)
{
    for (int i = 7; i >= 0; i--)
        putchar(((n >> i) & 1) ? '1' : '0');
}

void show_bit(const char *op, unsigned char a, unsigned char b, unsigned char r)
{
    print_bin(a);
    printf(" %s ", op);
    print_bin(b);
    printf(" = ");
    print_bin(r);
    printf("  (%d %s %d = %d)\n", a, op, b, r);
}

int main(void)
{
    int a = 17, b = 5;

    printf("=== Арифметичні оператори (a = %d, b = %d) ===\n", a, b);
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d (цілочисельне ділення)\n", a / b);
    printf("a / b = %.2f (дійсне ділення)\n", (float)a / b);
    printf("a %% b = %d (остача)\n", a % b);
    printf("-a    = %d (унарний мінус)\n", -a);

    int x = 10;
    printf("\nІнкремент і декремент (x = %d):\n", x);
    printf("x++ = %d, ", x++);
    printf("після цього x = %d\n", x);
    printf("++x = %d\n", ++x);
    printf("x-- = %d, ", x--);
    printf("після цього x = %d\n", x);
    printf("--x = %d\n", --x);

    printf("\n=== Оператори порівняння (a = %d, b = %d) ===\n", a, b);
    printf("a >  b : %d\n", a > b);
    printf("a >= b : %d\n", a >= b);
    printf("a <  b : %d\n", a < b);
    printf("a <= b : %d\n", a <= b);
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);
```

Результат виконання коду:

```
=== Арифметичні оператори (a = 17, b = 5) ===
a + b = 22
a - b = 12
a * b = 85
a / b = 3 (цілочисельне ділення)
a / b = 3.40 (дійсне ділення)
a % b = 2 (остача)
-a    = -17 (унарний мінус)

Інкремент і декремент (x = 10):
x++ = 10, після цього x = 11
++x = 12
x-- = 12, після цього x = 11
--x = 10

=== Оператори порівняння (a = 17, b = 5) ===
a >  b : 1
a >= b : 1
a <  b : 0
a <= b : 0
a == b : 0
a != b : 1

=== Логічні оператори (p = 1, q = 0) ===
p && q = 0 (І)
p || q = 1 (АБО)
!p     = 0 (НЕ)
!q     = 1 (НЕ)

=== Побітові оператори ===
m = 00001100 (12)
n = 00001010 (10)

00001100 & 00001010 = 00001000  (12 & 10 = 8)
00001100 | 00001010 = 00001110  (12 | 10 = 14)
00001100 ^ 00001010 = 00000110  (12 ^ 10 = 6)
~00001100 = 11110011  (~12 = 243)
00001100 >> 2 = 00000011  (12 >> 2 = 3)
00001100 << 2 = 00110000  (12 << 2 = 48)
```

Я написав програму мовою Сі, яка демонструє роботу арифметичних, порівняльних, логічних і побітових операторів. Для виведення результатів я підключив бібліотеку stdio.h і створив допоміжну функцію print_bin(), яка виводить число у двійковому форматі: в циклі вона зсуває число вправо та перевіряє кожен із 8 біт операцією & 1.

Спочатку я виконав арифметичні операції над числами 17 і 5: додавання, віднімання, множення, цілочисельне та дійсне ділення, остачу від ділення й унарний мінус. Також я показав префіксну та постфіксну форми інкременту й декременту. Вивід наочно показав, що x++ спочатку повертає старе значення, а

потім збільшує змінну, а ++x робить навпаки. Далі я продемонстрував оператори порівняння та логічні оператори &&, || і !, результати яких збіглися з таблицями істинності.

Для побітових операцій я взяв числа 12 (00001100) і 10 (00001010) та виконав над ними &, |, ^, ~, а також зсуви вправо й вліво на 2 розряди. Отримав відповідно 00001000, 00001110, 00000110, 11110011, 00000011 і 00110000. Для ~ і зсувів я використав приведення до unsigned char, бо інакше операнд розширюється до int і результат містив би зайві старші біти.

Завдання 3:

Частина коду програми:

```c
#include <stdio.h>

int main(void)
{
    int x;
    int *p;

    printf("Введіть число: ");
    scanf("%d", &x);

    p = &x;

    printf("\nЗначення змінної x      : %d\n", x);
    printf("Адреса змінної x        : %p\n", (void *)&x);
    printf("Значення вказівника p   : %p\n", (void *)p);
    printf("Значення через *p       : %d\n", *p);
    printf("Адреса самого вказівника: %p\n", (void *)&p);

    *p = *p + 1;
    printf("\nПісля *p = *p + 1 значення x = %d\n", x);
```

Результат виконання коду:

```
Введіть число: 312

Значення змінної x      : 312
Адреса змінної x        : 0x7fff48af3ebc
Значення вказівника p   : 0x7fff48af3ebc
Значення через *p       : 312
Адреса самого вказівника: 0x7fff48af3eb0

Після *p = *p + 1 значення x = 313


...Program finished with exit code 0
Press ENTER to exit console.
```

Я написав програму, у якій оголосив цілочисельну змінну x та вказівник p на цей тип. Число вводиться з клавіатури функцією scanf, якій передається адреса змінної &x. Потім я записав у вказівник адресу змінної операцією взяття адреси: p = &x.

Після цього програма виводить значення змінної x, її адресу та значення вказівника p. Вони збігаються, бо вказівник зберігає саме адресу змінної. Значення через вказівник отримується операцією розіменування *p, і воно дорівнює введеному числу. Я також вивів адресу самого вказівника, яка відрізняється від адреси x, бо вказівник теж є змінною і займає власне місце в пам'яті. Наприкінці я виконав *p = *p + 1 і переконався, що змінна x збільшилась на одиницю, тобто через вказівник можна змінювати значення змінної.

Завдання 4:

Код програми:

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    double a, b, c, d, x1, x2;

    printf("Розв'язання рівняння a*x^2 + b*x + c = 0\n");
    printf("Введіть коефіцієнти a, b, c: ");
    scanf("%lf %lf %lf", &a, &b, &c);

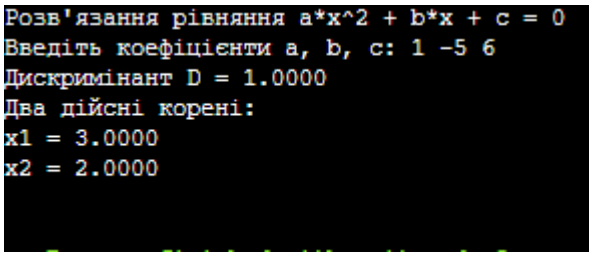
    if (a == 0)
    {
        if (b == 0)
        {
            if (c == 0)
                printf("Рівняння має безліч розв'язків.\n");
            else
                printf("Рівняння не має розв'язків.\n");
        }
        else
        {
            x1 = -c / b;
            printf("Це лінійне рівняння, x = %.4f\n", x1);
        }
        return 0;
    }

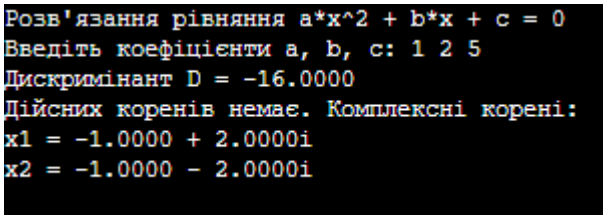
    d = b * b - 4 * a * c;
    printf("Дискримінант D = %.4f\n", d);

    if (d > 0)
    {
        x1 = (-b + sqrt(d)) / (2 * a);
        x2 = (-b - sqrt(d)) / (2 * a);
        printf("Два дійсні корені:\n");
        printf("x1 = %.4f\n", x1);
        printf("x2 = %.4f\n", x2);
    }
    else if (d == 0)
    {
        x1 = -b / (2 * a);
        printf("Один корінь (кратності 2): x = %.4f\n", x1);
    }
    else
    {
        double re = -b / (2 * a);
        double im = sqrt(-d) / (2 * a);
        printf("Дійсних коренів немає. Комплексні корені:\n");
        printf("x1 = %.4f + %.4fi\n", re, fabs(im));
        printf("x2 = %.4f - %.4fi\n", re, fabs(im));
    }

    return 0;
}
```

Результат виконання коду:





Я написав програму, яка розв'язує квадратне рівняння виду $ax^2 + bx + c = 0$, коефіцієнти якого вводяться з клавіатури. Для обчислень я використав тип double і підключив бібліотеку math.h, з якої взяв функції sqrt (квадратний корінь) та fabs (модуль).

Спочатку програма перевіряє, чи коефіцієнт a дорівнює нулю. Якщо так, рівняння стає лінійним: я розв'язую його як $x = -c / b$, а у випадках $b = 0$ виводжу повідомлення про безліч розв'язків (при $c = 0$) або про їх відсутність. Інакше програма обчислює дискримінант за формулою $D = b^2 - 4ac$ і залежно від його знака вибирає гілку розв'язку. Якщо $D > 0$, то є два дійсні корені $x = (-b \pm \sqrt{D}) / 2a$. Якщо $D = 0$, то є один корінь $x = -b / 2a$. Якщо $D < 0$, то дійсних коренів немає, і я виводжу комплексні корені, обчислюючи дійсну частину $-b / 2a$ та уявну $\sqrt{(-D)} / 2a$.

Для перевірки я ввів кілька наборів коефіцієнтів. Для рівняння $x^2 - 5x + 6 = 0$ програма знайшла корені 3 і 2, для $x^2 - 4x + 4 = 0$ один корінь 2, а для $x^2 + 2x + 5 = 0$ комплексні корені $-1 \pm 2i$. Усі результати збігаються з обчисленими вручну.

Висновок:

Під час виконання лабораторної роботи я вивчив основні типи змінних мови Сі, операції над ними та їх пріоритет, а також познайомився з вказівниками й бібліотекою math.h. На практиці я опрацював арифметичні, порівняльні, логічні та побітові оператори. Подання чисел у двійковому форматі допомогло мені наочно побачити, як побітові операції діють на окремі розряди, а також зрозуміти різницю між префіксною й постфіксною формами інкременту, між цілочисельним і дійсним діленням, між логічними та побітовими операторами.

Працюючи з вказівниками, я навчився отримувати адресу змінної операцією & та звертатися до її значення операцією розіменування *. Я переконався, що через

вказівник можна не лише читати, а й змінювати значення змінної, а це важливо для передачі параметрів у функції та роботи з масивами й динамічними структурами даних.

Розв'язуючи квадратне рівняння, я закріпив уміння використовувати умовні оператори та математичні функції sqrt і fabs, а також навчився враховувати всі можливі випадки: різні знаки дискримінанта й виродження рівняння в лінійне. Усі програми я скомпілював і перевірив на різних вхідних даних, результати збіглися з очікуваними.